// Security Assessment

12.08.2025 - 12.22.2025

OrderBook EVM & Solana

MO

HALBORN

OrderBook EVM & Solana - M0

Prepared by: HALBORN

Last Updated 01/14/2026

Date of Engagement: December 8th, 2025 - December 22nd, 2025

Summary

100% OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
4	0	0	0	2	2

TABLE OF CONTENTS

1. Introduction

`M0` engaged Halborn to conduct a security assessment on their `OrderBook` EVM contract and `order_book` Solana program beginning on December 8, 2025 and ending on December 22, 2025. The security assessment was scoped to the smart contracts provided in the **liquidity-delivery repository**, commit hashes, and further details can be found in the Scope section of this report.

The `M0 team` is releasing both `OrderBook` EVM contract and `order_book` Solana program. Combined, this protocol enables users to create limit orders for token swaps, supporting both local (same-chain) and cross-chain order fulfillment through a solver network that fills orders and receives proportional token releases.

2. Assessment Summary

Halborn was provided 11 business days for the engagement and assigned one full-time security engineer to review the security of the Solana program and EVM contract in scope. The engineer is a blockchain and smart contract security expert with advanced smart contract hacking skills, and deep knowledge of multiple blockchain protocols.

The purpose of the assessment is to:

- Identify potential security issues within the Solana program and EVM contract.
- Ensure that smart contract functionality operates as intended.
- Verify the correctness of cross-chain order fulfillment and token release mechanisms.
- Review access control and administrative functions for proper authorization.

In summary, Halborn identified some improvements to reduce the likelihood and impact of risks, which were mostly addressed by the `M0 team`. The main ones were the following:

- `Add validation to ensure amount_in_to_release is non-zero in partial fills to prevent solver fund loss.`
- `Implement a maximum cap for the finality_buffer parameter to prevent extended fund lockup from admin misconfiguration.`
- `Add zero address validation for messenger parameter in OrderBook constructor.`
- `Fix the FillReported event to emit the correct amount_in_to_release value on full fills.`

3. Test Approach And Methodology

Halborn performed a combination of manual review and security testing based on scripts to balance efficiency, timeliness, practicality, and accuracy in regard to the scope of this assessment. While manual testing is recommended to uncover flaws in logic, process, and implementation; automated testing techniques help enhance coverage of the code and can quickly identify items that do not follow the security best practices. The following phases and associated tools were used during the assessment:

- Research into architecture and purpose.
- Differences analysis using GitLens to have a proper view of the differences between the mentioned commits
- Graphing out functionality and programs logic/connectivity/functions along with state changes

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets** of **Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker's control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N) Low (C:L) Medium (C:M) High (C:H) Critical (C:C)	0 0.25 0.5 0.75 1
Integrity (I)	None (I:N) Low (I:L) Medium (I:M) High (I:H) Critical (I:C)	0 0.25 0.5 0.75 1
Availability (A)	None (A:N) Low (A:L) Medium (A:M) High (A:H) Critical (A:C)	0 0.25 0.5 0.75 1
Deposit (D)	None (D:N) Low (D:L) Medium (D:M) High (D:H) Critical (D:C)	0 0.25 0.5 0.75 1
Yield (Y)	None (Y:N) Low (Y:L) Medium (Y:M) High (Y:H) Critical (Y:C)	0 0.25 0.5 0.75 1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

SEVERITY COEFFICIENT (C)	COEFFICIENT VALUE	NUMERICAL VALUE
Reversibility (r)	None (R:N) Partial (R:P) Full (R:F)	1 0.5 0.25
Scope (s)	Changed (S:C) Unchanged (S:U)	1.25 1

Severity Coefficient C is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score S is obtained by:

$$S = min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4

SEVERITY	SCORE VALUE RANGE
Informational	0 - 1.9

5. SCOPE

REPOSITORY

(a) Repository: liquidity-delivery

(b) Assessed Commit ID: 15a972b

(c) Items in scope:

- evm/src/OrderBook.sol
- evm/src/interfaces/IOrderBook.sol
- svm/programs/order_book/src/constants.rs
- svm/programs/order_book/src/error.rs
- svm/programs/order_book/src/instructions/admin.rs
- svm/programs/order_book/src/instructions/claim_refund.rs
- svm/programs/order_book/src/instructions/configure_destination.rs
- svm/programs/order_book/src/instructions/fill.rs
- svm/programs/order_book/src/instructions/initialize.rs
- svm/programs/order_book/src/instructions/mod.rs
- svm/programs/order_book/src/instructions/open.rs
- svm/programs/order_book/src/instructions/report_fill.rs
- svm/programs/order_book/src/instructions/request_cancel.rs
- svm/programs/order_book/src/lib.rs
- svm/programs/order_book/src/state/mod.rs
- svm/programs/order_book/src/state/orders.rs
- svm/programs/order_book/src/utils.rs

Out-of-Scope: Third party dependencies and economic attacks.

REMEDIATION COMMIT ID:

- https://github.com/m0-foundation/liquidity-delivery/pull/7/changes#diff-223d97547c1ca006ccfc30e9a39ede58c7c298cf21d74f3bff4f2b276da82866
- https://github.com/m0-foundation/liquidity-delivery/pull/30
- https://github.com/m0-foundation/liquidity-delivery/pull/24

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW

CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
0	0	0	2	2

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
MISSING ZERO AMOUNT VALIDATION IN PARTIAL FILL ALLOWS SOLVER FUND LOSS	LOW	RISK ACCEPTED - 01/09/2026
MISSING MAXIMUM CAP FOR FINALITY BUFFER ALLOWS EXTENDED FUND LOCKUP FOR BOTH EVM AND SVM	LOW	SOLVED - 01/09/2026
MISSING ZERO ADDRESS VALIDATION FOR MESSENGER IN ORDERBOOK CONSTRUCTOR	INFORMATIONAL	SOLVED - 01/09/2026
SOLANA REPORT_ORDER_FILL INSTRUCTION'S FILLREPORTED EVENT EMITS INCORRECT AMOUNT ON FULL FILL	INFORMATIONAL	SOLVED - 01/09/2026

7. FINDINGS & TECH DETAILS

7.1 MISSING ZERO AMOUNT VALIDATION IN PARTIAL FILL ALLOWS SOLVER FUND LOSS

// LOW

Description

The `fill_native_order` and `fill_foreign_order` instructions (Solana) and the `_fillOrder` function (EVM) allow solvers to fill orders either fully or partially.

When a partial fill occurs, the solver specifies `amount_out_to_fill` (tokens to send to the recipient), and `amount_in_to_release` (tokens the solver receives) is calculated proportionally.

However, both implementations use integer division which rounds down and do not validate that the solver will receive a non-zero amount, as shown in the code snippets below.

svm/programs/order_book/src/instructions/fill.rs

```
188 } else {
189     // Calculate the amount in to release based on the proportion of amount out being filled
190     let amount_in_to_release: u64 = (fill_params.amount_out_to_fill as u128)
191         .checked_mul(order.amount_in)
192         .ok_or(OrderBookError::MathOverflow)?
193         .checked_div(order.amount_out)
194         .ok_or(OrderBookError::MathUnderflow)?
195         .try_into()
196         .map_err(|_| OrderBookError::MathOverflow)?;
197
198     (amount_in_to_release, fill_params.amount_out_to_fill)
199 };
```

evm/src/OrderBook.sol

```
425 // Calculate the corresponding of token in to release to the filler
426 uint128 amountInRemaining_ = orderData_.amountIn - filledAmounts.amountInReleased;
427 uint128 amountInToRelease_ = fullFill_
428     ? amountInRemaining_
429     : ((uint256(orderData_.amountIn) * amountOutToFill_) / orderData_.amountOut).toUint128();
```

When the ratio `amount_in / amount_out` is small enough, and the partial fill amount is chosen such that `(amount_out_to_fill * amount_in) / amount_out` evaluates to zero, the solver transfers their `token_out` to the recipient but receives zero `token_in` in return.

A solver unaware of this rounding behavior could attempt a small partial fill and lose funds. The solver would transfer their `token_out` to the order's recipient while receiving nothing in exchange.

This results in direct financial loss to the solver with no mechanism for recovery.

Proof of Concept

POC Code

```
#[test]
fn fill_native_order_small_fill_rounding_to_zero_amount_in() -> Result<(), Box<dyn Error>> {
    let mut test = OrderBookTest::new()?;
    test.initialize()?;

    // ================================================================================
    // TEST SETUP
    // ================================================================================
    println!("\n=== TEST SETUP ===");

    // Create an order with a large amount_out relative to amount_in
    // This makes small fills result in amount_in_to_release = 0 due to integer division
    // amount_in_to_release = (amount_out_to_fill * amount_in) / amount_out
    // If amount_out_to_fill = 100 and amount_in = 1_000, amount_out = 1_000_000
    // Then: amount_in_to_release = (100 * 1_000) / 1_000_000 = 0
    let sender = test.get_user("alice");
    let solver = test.get_user("solver");
    let token_in_mint = test.get_mint("token-in-spl-6");

    let order_params = order_book::instructions::open::OrderParams {
        dest_chain_id: CHAIN_ID,
        fill_deadline: test.ctx.svm.get_sysvar::<Clock>().unix_timestamp as u64 + 86400,
        token_out: test.get_mint("token-out-spl-6").to_bytes(),
        amount_in: 1_000,              // Small amount_in
        amount_out: 1_000_000,         // Large amount_out (1000x ratio)
        recipient: sender.pubkey().to_bytes(),
        solver: solver.pubkey().to_bytes(),
    };

    println!("Order parameters:");
    println!("   - amount_in: {} (tokens sender deposits)", order_params.amount_in);
    println!("   - amount_out: {} (tokens sender wants)", order_params.amount_out);
    println!("   - ratio: 1:{}", order_params.amount_out / order_params.amount_in as u128);

    let order_id = test.open_order("alice", "token-in-spl-6", &order_params)?;
    println!("   - Order created by Alice");

    // Solver fills with a small amount that would result in amount_in_to_release = 0
    let small_fill_amount: u64 = 100;
    let expected_amount_in_released = (small_fill_amount as u128 * order_params.amount_in as u128)
        / order_params.amount_out as u128;

    println!("   - Solver will fill with: {} token_out", small_fill_amount);
    println!("   - Expected amount_in_to_release: ({} * {}) / {} = {}",
        small_fill_amount, order_params.amount_in, order_params.amount_out, expected_amount_in_relea

    // Get token account addresses
    let alice_token_out_ata = test.get_ata("token-out-spl-6", "alice");
    let solver_token_in_ata = test.get_ata("token-in-spl-6", "solver");
    let solver_token_out_ata = test.get_ata("token-out-spl-6", "solver");
    let order_account = test
        .ctx
        .svm
        .get_pda(&[ORDER_SEED_PREFIX, &order_id], &order_book::ID);
    let order_token_in_ata = get_associated_token_address(&order_account, &token_in_mint);

    // ================================================================================
    // PRE-CALL ASSERTIONS
    // ================================================================================
    println!("\n=== PRE-CALL ASSERTIONS ===");

    // --------------------------------------------------------------------------------
    // 1. SOLVER BALANCES
    // --------------------------------------------------------------------------------
    println!("1. Solver Balances:");
    let solver_token_in_balance_before = test.get_token_balance(&solver_token_in_ata)?;
```

```rust
let solver_token_out_balance_before = test.get_token_balance(&solver_token_out_ata)?;
println!("   - Solver token_in balance: {}", solver_token_in_balance_before);
println!("   - Solver token_out balance: {}", solver_token_out_balance_before);
assert!(solver_token_out_balance_before >= small_fill_amount, "solver should have enough token_o
println!("   [PASS] Solver has enough token_out to fill");

// ------------------------------------------------------------------------
// 2. RECIPIENT BALANCES
// ------------------------------------------------------------------------
println!("\n2. Recipient (Alice) Balances:");
let alice_token_out_balance_before = test.get_token_balance(&alice_token_out_ata)?;
println!("   - Alice token_out balance: {}", alice_token_out_balance_before);

// ------------------------------------------------------------------------
// 3. ORDER STATE
// ------------------------------------------------------------------------
println!("\n3. Order State:");
let order_token_in_balance_before = test.get_token_balance(&order_token_in_ata)?;
let (_, order_data_before) = test.get_native_order_account(&order_id)?;
println!("   - Order token_in balance: {}", order_token_in_balance_before);
println!("   - Order status: {:?}", order_data_before.data.status);
println!("   - amount_out_filled: {}", order_data_before.data.amount_out_filled);
println!("   - amount_in_released: {}", order_data_before.data.amount_in_released);
assert_eq!(order_data_before.data.amount_out_filled, 0, "no amount should be filled yet");
assert_eq!(order_data_before.data.amount_in_released, 0, "no amount should be released yet");
println!("   [PASS] Order is in correct initial state");

println!("\n=== PRE-CALL ASSERTIONS PASSED ===");

// ============================================================================
// EXECUTE FILL
// ============================================================================
println!("\n=== EXECUTING FILL ===");
println!("Solver filling order with {} token_out...", small_fill_amount);

let fill_params = order_book::instructions::fill::FillParams {
    amount_out_to_fill: small_fill_amount,
    origin_recipient: solver.pubkey().to_bytes(),
};

let ix = test.create_fill_native_order_ix(&solver.pubkey(), order_id, &fill_params)?;
test.ctx
    .execute_instruction(ix, &[&solver])?
    .assert_success();

println!("[PASS] Fill executed successfully");

// ============================================================================
// POST-CALL ASSERTIONS
// ============================================================================
println!("\n=== POST-CALL ASSERTIONS ===");

// ------------------------------------------------------------------------
// 1. SOLVER TOKEN BALANCES - DEMONSTRATING THE VULNERABILITY
// ------------------------------------------------------------------------
println!("1. Solver Token Balances (Vulnerability Demonstration):");
let solver_token_in_balance_after = test.get_token_balance(&solver_token_in_ata)?;
let solver_token_out_balance_after = test.get_token_balance(&solver_token_out_ata)?;

let token_in_received = solver_token_in_balance_after - solver_token_in_balance_before;
let token_out_spent = solver_token_out_balance_before - solver_token_out_balance_after;

println!("   - Token_in received by solver: {}", token_in_received);
println!("   - Token_out spent by solver: {}", token_out_spent);

assert_eq!(token_in_received, expected_amount_in_released as u64, "solver should receive calcula
assert_eq!(token_out_spent, small_fill_amount, "solver should have spent token_out");

if token_in_received == 0 && token_out_spent > 0 {
    println!("   [VULNERABILITY] Solver spent {} token_out but received 0 token_in!", token_out_
}

// ------------------------------------------------------------------------
```

```
        // 2. RECIPIENT BALANCES
        // ------------------------------------------------------------------------
        println!("\n2. Recipient (Alice) Balances:");
        let alice_token_out_balance_after = test.get_token_balance(&alice_token_out_ata)?;
        let token_out_received = alice_token_out_balance_after - alice_token_out_balance_before;

        println!("   - Token_out received by Alice: {}", token_out_received);
        assert_eq!(token_out_received, small_fill_amount, "alice should receive token_out from solver");
        println!("   [PASS] Alice received the token_out from solver");

        // ------------------------------------------------------------------------
        // 3. ORDER STATE UPDATED
        // ------------------------------------------------------------------------
        println!("\n3. Order State After Fill:");
        let (_, order_data_after) = test.get_native_order_account(&order_id)?;

        println!("   - amount_out_filled: {}", order_data_after.data.amount_out_filled);
        println!("   - amount_in_released: {}", order_data_after.data.amount_in_released);

        assert_eq!(order_data_after.data.amount_out_filled, small_fill_amount as u128, "amount_out_fille
        assert_eq!(order_data_after.data.amount_in_released, expected_amount_in_released, "amount_in_rel

        if order_data_after.data.amount_in_released == 0 {
            println!("   [VULNERABILITY] Order progress updated but no token_in released!");
        }

        // ------------------------------------------------------------------------
        // 4. VULNERABILITY SUMMARY
        // ------------------------------------------------------------------------
        println!("\n4. Vulnerability Summary:");
        println!("   - Solver sent: {} token_out to recipient", token_out_spent);
        println!("   - Solver received: {} token_in (should be proportional)", token_in_received);
        println!("   - Calculation: ({} * {}) / {} = {}",
            small_fill_amount, order_params.amount_in, order_params.amount_out, expected_amount_in_relea

        if token_in_received == 0 {
            println!("   [IMPACT] Solver loses funds due to integer division rounding to zero");
        }

        println!("\n=== ALL POST-CALL ASSERTIONS PASSED ===");

        // ============================================================================
        // END POST-CALL ASSERTIONS
        // ============================================================================

        Ok(())
    }
```

Evidence

```
=== TEST SETUP ===
Order parameters:
   - amount_in: 1000 (tokens sender deposits)
   - amount_out: 1000000 (tokens sender wants)
   - ratio: 1:1000
   - Order created by Alice
   - Solver will fill with: 100 token_out
   - Expected amount_in_to_release: (100 * 1000) / 1000000 = 0

=== PRE-CALL ASSERTIONS ===
1. Solver Balances:
   - Solver token_in balance: 10000000000
   - Solver token_out balance: 10000000000
   [PASS] Solver has enough token_out to fill

2. Recipient (Alice) Balances:
   - Alice token_out balance: 10000000000

3. Order State:
   - Order token_in balance: 1000
   - Order status: Created
   - amount_out_filled: 0
   - amount_in_released: 0
   [PASS] Order is in correct initial state

=== PRE-CALL ASSERTIONS PASSED ===

=== EXECUTING FILL ===
Solver filling order with 100 token_out...
[PASS] Fill executed successfully

=== POST-CALL ASSERTIONS ===
1. Solver Token Balances (Vulnerability Demonstration):
   - Token_in received by solver: 0
   - Token_out spent by solver: 100
   [VULNERABILITY] Solver spent 100 token_out but received 0 token_in!

2. Recipient (Alice) Balances:
   - Token_out received by Alice: 100
   [PASS] Alice received the token_out from solver

3. Order State After Fill:
   - amount_out_filled: 100
   - amount_in_released: 0
   [VULNERABILITY] Order progress updated but no token_in released!

4. Vulnerability Summary:
   - Solver sent: 100 token_out to recipient
   - Solver received: 0 token_in (should be proportional)
   - Calculation: (100 * 1000) / 1000000 = 0
   [IMPACT] Solver loses funds due to integer division rounding to zero

=== ALL POST-CALL ASSERTIONS PASSED ===
test instructions::fill::edge_cases::fill_native_order_small_fill_rounding_to_zero_amount_in ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 102 filtered out; finished in 0.17s
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:L/Y:N (3.1)

Recommendation

It is recommended to add validation that rejects partial fills where `amount_in_to_release` rounds to zero. This ensures that every partial fill releases a proportional amount of tokens to the solver. Proposal of fixed code:

svm/programs/order_book/src/instructions/fill.rs

```
let (amount_in_to_release, amount_out_to_fill): (u64, u64) = if full_fill {
    // Set the order status to completed
    order.status = OrderStatus::Completed;

    // Set the fill amount out to the remaining amount
    // The amount in to release is the remaining amount in the order ATA
    // Any extra tokens are considered a donation to the solver that completes the order
    require!(
        ctx.accounts.order_token_in_ata.amount
            >= amount_in_remaining
                .try_into()
                .map_err(|_| OrderBookError::InvalidFillAmount)?,
        OrderBookError::InvalidFillAmount
    );
    (
        ctx.accounts.order_token_in_ata.amount,
        amount_out_remaining
            .try_into()
            .map_err(|_| OrderBookError::InvalidFillAmount)?,
    )
} else {
    // Calculate the amount in to release based on the proportion of amount out being filled
    let amount_in_to_release: u64 = (fill_params.amount_out_to_fill as u128)
        .checked_mul(order.amount_in)
        .ok_or(OrderBookError::MathOverflow)?
        .checked_div(order.amount_out)
        .ok_or(OrderBookError::MathUnderflow)?
        .try_into()
        .map_err(|_| OrderBookError::MathOverflow)?;

    (amount_in_to_release, fill_params.amount_out_to_fill)
};

// Reject fills that result in zero token release
require!(amount_in_to_release > 0, OrderBookError::InvalidFillAmount);
```

evm/src/OrderBook.sol

```
// Calculate the corresponding of token in to release to the filler
uint128 amountInRemaining_ = orderData_.amountIn - filledAmounts.amountInReleased;
uint128 amountInToRelease_ = fullFill_
    ? amountInRemaining_
    : ((uint256(orderData_.amountIn) * amountOutToFill_) / orderData_.amountOut).toUint128();

// Reject fills that result in zero token release
if (amountInToRelease_ == 0) revert InvalidFillAmount();
```

Remediation Comment

RISK ACCEPTED: The **MO team** accepted the risk of this finding, because in their primary use case (stablecoin-to-stablecoin transfers with similar prices), it only affects very small amounts. Implementing the fix would create edge cases where orders cannot be fully filled, preventing finalization and causing dust accumulation in the contract.

7.2 MISSING MAXIMUM CAP FOR FINALITY BUFFER ALLOWS EXTENDED FUND LOCKUP FOR BOTH EVM AND SVM

// LOW

Description

The `configure_destination` (Solana) and `setDestinationConfig` (EVM) functions allow the admin to configure destination chains for cross-chain orders, including setting a `finality_buffer/finalityBuffer` parameter that determines how long users must wait after order expiration before claiming refunds.

The validation logic checks that the value is provided and non-zero when enabling support for a destination, but there is no maximum cap validation for this parameter in either implementation.

svm/programs/order_book/src/instructions/configure_destination.rs

```
35 impl ConfigureDestination<'_> {
36     fn validate(&self, dest_chain_id: u32, is_supported: bool, finality_buffer: Option<u64>) -> R
37         if dest_chain_id == self.global_account.chain_id {
38             return err!(OrderBookError::InvalidDestChainId);
39         }
40 
41         if is_supported {
42             if finality_buffer.is_none() || finality_buffer.unwrap() == 0 {
43                 return err!(OrderBookError::InvalidFinalityBuffer);
44             }
45             // No maximum cap validation exists here
46         }
47         Ok(())
48     }
49     ...
50 }
```

evm/src/OrderBook.sol

```
517 function setDestinationConfig(
518     uint32 destChainId_,
519     bool isSupported_,
520     uint32 finalityBuffer_
521 ) external override onlyRole(DEFAULT_ADMIN_ROLE) {
522     if (destChainId_ == chainId) revert InvalidDestinationChain();
523     if (isSupported_ && finalityBuffer_ == 0) revert InvalidFinalityBuffer();
524     // No maximum cap validation exists here
525     ...
526 }
```

An admin could accidentally set an extremely high finalityBuffer value (e.g., 1 year). This would lock user funds for extended periods since `claim_refund` / `claimRefund` requires waiting `fill_deadline + finality_buffer` or `cancel_requested_at + finality_buffer`.

The situation cannot be quickly corrected because reducing the buffer is delayed by the old (high) buffer value, effectively trapping the system in a broken state for the duration of the misconfigured buffer.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:C/I:C/D:C/Y:N (3.0)

Recommendation

It is recommended to add a maximum cap validation for the `finalityBuffer` parameter in both implementations.

The `M0 team` should determine a prudent maximum value that prevents accidental misconfigurations while still allowing flexibility for different chain finality requirements.

svm/programs/order_book/src/instructions/configure_destination.rs

```
35  const MAX_FINALITY_BUFFER: u64 = 7 * 24 * 60 * 60; // 7 days in seconds
36
37  impl ConfigureDestination<'_> {
38      fn validate(&self, dest_chain_id: u32, is_supported: bool, finality_buffer: Option<u64>) -> R
39          ...
40          if is_supported {
41              if finality_buffer.is_none() || finality_buffer.unwrap() == 0 {
42                  return err!(OrderBookError::InvalidFinalityBuffer);
43              }
44              if finality_buffer.unwrap() > MAX_FINALITY_BUFFER {
45                  return err!(OrderBookError::FinalityBufferTooLarge);
46              }
47          }
48          Ok(())
49      }
50      ...
51  }
```

evm/src/OrderBook.sol

```
517 uint32 constant MAX_FINALITY_BUFFER = 7 days;
518
519 function setDestinationConfig(
520     uint32 destChainId_,
521     bool isSupported_,
522     uint32 finalityBuffer_
523 ) external override onlyRole(DEFAULT_ADMIN_ROLE) {
524     if (destChainId_ == chainId) revert InvalidDestinationChain();
525     if (isSupported_ && finalityBuffer_ == 0) revert InvalidFinalityBuffer();
526     if (isSupported_ && finalityBuffer_ > MAX_FINALITY_BUFFER) revert FinalityBufferTooLarge();
527     ...
528 }
```

Remediation Comment

SOLVED: The **M0 team** solved the issue by deleting the finalization buffer feature.

The change was included in the PR mentioned below, but the files in scope for this change are only:

- *svm/programs/order_book/src/instructions/configure_destination.rs*
- *svm/programs/order_book/src/instructions/destination.rs*

- *evm/src/OrderBook.sol* (the `setDestinationConfig` function removed the reference to the finality buffer feature)

Remediation Hash

https://github.com/m0-foundation/liquidity-delivery/pull/7/changes#diff-223d97547c1ca006ccfc30e9a39ede58c7c298cf21d74f3bff4f2b276da82866

7.3 MISSING ZERO ADDRESS VALIDATION FOR MESSENGER IN ORDERBOOK CONSTRUCTOR

// INFORMATIONAL

Description

The `OrderBook` contract constructor accepts a `messenger_` parameter that is assigned directly to the immutable `messenger` state variable without validating that it is not the zero address.

As shown in the code snippet below, the constructor does not include any validation for the `messenger_` parameter.

evm/src/OrderBook.sol

```
69  constructor(uint32 chainId_, address messenger_) {
70      chainId = chainId_;
71      messenger = messenger_;
72  }
```

Since `messenger` is an immutable variable, it cannot be changed after deployment. If the contract is deployed with `messenger_` set to `address(0)`, all cross-chain operations that rely on the messenger will fail or behave unexpectedly.

Deploying the contract with a zero address messenger would result in a non-functional contract for cross-chain operations:

1. The `_fillOrder` function calls `IMessenger(messenger).sendFillReport()` for cross-chain fills, which would revert when calling the zero address.
2. The `reportFill` function validates that `msg.sender == messenger`, making it impossible for anyone to call this function since no account can be `address(0)`.

This would require redeployment of the entire contract to fix, potentially causing additional gas costs.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:H/I:H/D:N/Y:N (1.9)

Recommendation

It is recommended to add a zero address check in the constructor for the `messenger_` parameter to prevent deployment with an invalid messenger address.

evm/src/OrderBook.sol

```
69  constructor(uint32 chainId_, address messenger_) {
70      if (messenger_ == address(0)) revert InvalidAddress();
71      chainId = chainId_;
72      messenger = messenger_;
73  }
```

Remediation Comment

SOLVED: The **M0 team** solved the issue by implementing the suggested changes. The name of the `messenger` parameter was changed to `portal`, but functionally it is the same parameter.

Remediation Hash

https://github.com/m0-foundation/liquidity-delivery/pull/30

7.4 SOLANA REPORT_ORDER_FILL INSTRUCTION'S FILLREPORTED EVENT EMITS INCORRECT AMOUNT ON FULL FILL

// INFORMATIONAL

Description

The `report_order_fill` instruction calculates the actual `amount_in_to_release` based on whether it's a full fill or partial fill.

For full fills, the actual transfer amount is determined by the order's token account balance `order_token_in_ata.amount`, which may include additional tokens donated to the solver.

However, as shown in the code snippet below, the `FillReported` event emits `fill_report.amount_in_to_release` instead of the calculated `amount_in_to_release` variable that represents the actual transferred amount.

svm/programs/order_book/src/instructions/report_fill.rs

```
131 let amount_in_to_release: u64 = if full_fill {
132     // Any tokens sent to this account after the order is created are donated to the solver
133     ctx.accounts.order_token_in_ata.amount
134 } else {
135     fill_report
136         .amount_in_to_release
137         .try_into()
138         .map_err(|_| OrderBookError::MathOverflow)?
139 };
140
141 // Transfer the input tokens from the order to the designated recipient
142 transfer_tokens_from_program(
143     ...
144     amount_in_to_release,
145     ...
146 )?;
147
148 // Emit an event for the fill report
149 emit_cpi!(FillReported {
150     order_id: fill_report.order_id,
151     amount_in_to_release: fill_report.amount_in_to_release,  // Uses fill_report value, not actua
152     amount_out_filled: fill_report.amount_out_filled,
153     origin_recipient: fill_report.origin_recipient,
154 });
```

Off-chain systems and indexers relying on the `FillReported` event will receive incorrect data about the actual token amount transferred on full fills.

This data inconsistency can lead to incorrect accounting in off-chain tracking systems, discrepancies between on-chain state and indexed data, and potential issues for solvers tracking their received amounts.

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:N (0.0)

Recommendation

It is recommended to emit the actual calculated `amount_in_to_release` value in the `FillReported` event instead of the `fill_report.amount_in_to_release` value.

This ensures the event accurately reflects the tokens transferred on-chain.

svm/programs/order_book/src/instructions/report_fill.rs

```
156 let amount_in_to_release: u64 = if full_fill {
157     // Any tokens sent to this account after the order is created are donated to the solver
158     ctx.accounts.order_token_in_ata.amount
159 } else {
160     fill_report
161         .amount_in_to_release
162         .try_into()
163         .map_err(|_| OrderBookError::MathOverflow)?
164 };
165
166 // Transfer the input tokens from the order to the designated recipient
167 transfer_tokens_from_program(
168     ...
169     amount_in_to_release,
170     ...
171 )?;
172
173 // Emit an event for the fill report
174 emit_cpi!(FillReported {
175     order_id: fill_report.order_id,
176     amount_in_to_release: amount_in_to_release as u128,  // Use actual calculated amount
177     amount_out_filled: fill_report.amount_out_filled,
178     origin_recipient: fill_report.origin_recipient,
179 });
```

Remediation Comment

SOLVED: The **M0 team** solved the issue by implementing the suggested changes.

Remediation Hash

https://github.com/m0-foundation/liquidity-delivery/pull/24

8. AUTOMATED TESTING

Cargo Audit (Solana)

Halborn used `cargo-audit`, a security scanner for vulnerabilities reported to the RustSec Advisory Database, to analyze the Solana program dependencies. All vulnerabilities published in **https://crates.io** are stored in a repository named The RustSec Advisory Database. cargo audit is a human-readable version of the advisory database which performs a scanning on Cargo.lock. Security Detections are only in scope. All vulnerabilities shown here were already disclosed in the above report. However, to better assist the developers maintaining this code, the reviewers are including the output with the dependencies tree, and this is included in the cargo audit output to better know the dependencies affected by unmaintained and vulnerable crates.

Results

ID	Package	Short Description
RUSTSEC-2024-0344	curve25519-dalek	Timing variability in curve25519-dalek's
RUSTSEC-2022-0093	ed25519-dalek	Double Public Key Signing Function Oracle Attack on `ed25519-dalek`

Slither Static Analysis (EVM)

Halborn also used `Slither`, a static analysis framework for Solidity, to detect common vulnerabilities and code quality issues in the EVM contracts.

Results

Detector	Location	Description	Assessment
missing-zero-check	OrderBook.constructor	messenger_ parameter lacks zero-address validation	Informational (Confirmed)
reentrancy-benign	openOrderWithPermit functions	State changes after external permit() call	False Positive
reentrancy-events	_fillOrder	Event emitted after external messenger.sendFillReport() call	False Positive
dangerous-strict-equality	_requestCancelOrder	Strict equality comparison on destChainId	False Positive
timestamp	Multiple functions	Usage of block.timestamp for deadline comparisons	False Positive

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.